

10-14-25 Research Progress

Summary of Aug – Sept Progress

J.C. Vaught

Undergraduate Assistants: Samuel Cancilla and Nathan Kirk
Department of Mechanical Engineering

Research Summary

Graduate Student

J.C. Vaught – M.S. Student Mechanical Engineering

- Machine learning infrastructure development
- GPU sharding and distributed training optimization
- 3 Pulse Length MAE preliminary results

Undergraduate Research Assistants

Samuel Cancilla, 2nd year

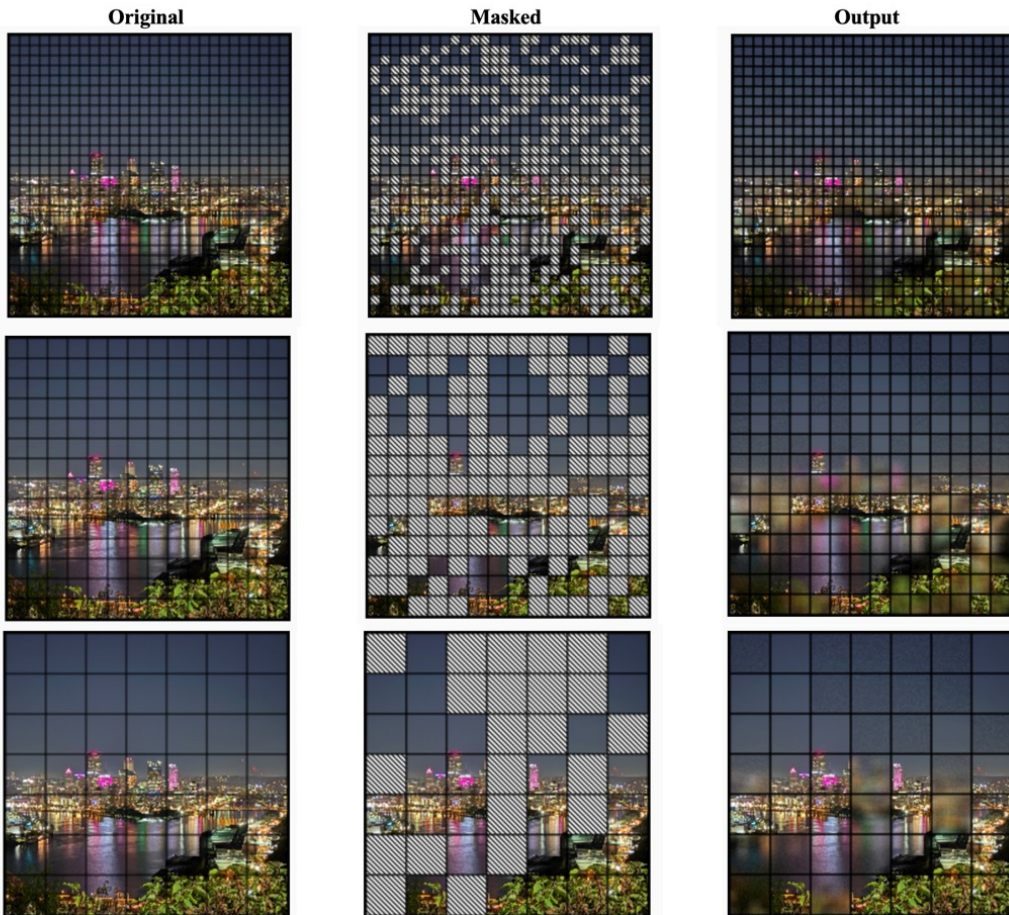
- ST-DBSCAN implementation
- Object tracking algorithms- not included

Nathan Kirk, 4th year

- CFAR algorithm development- not included
- Data collection and processing- not included

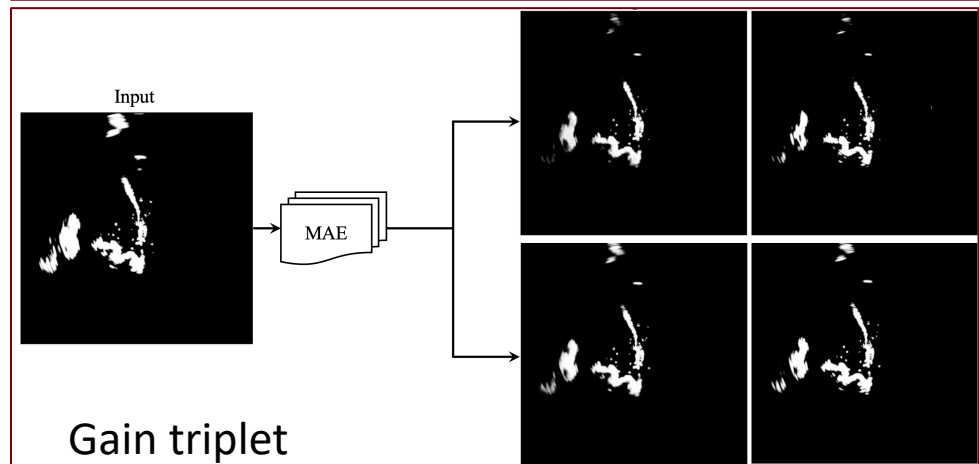
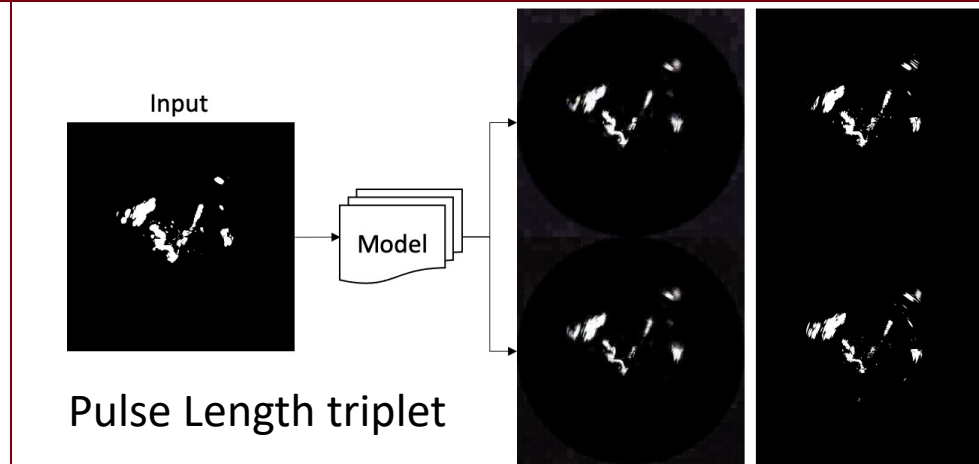
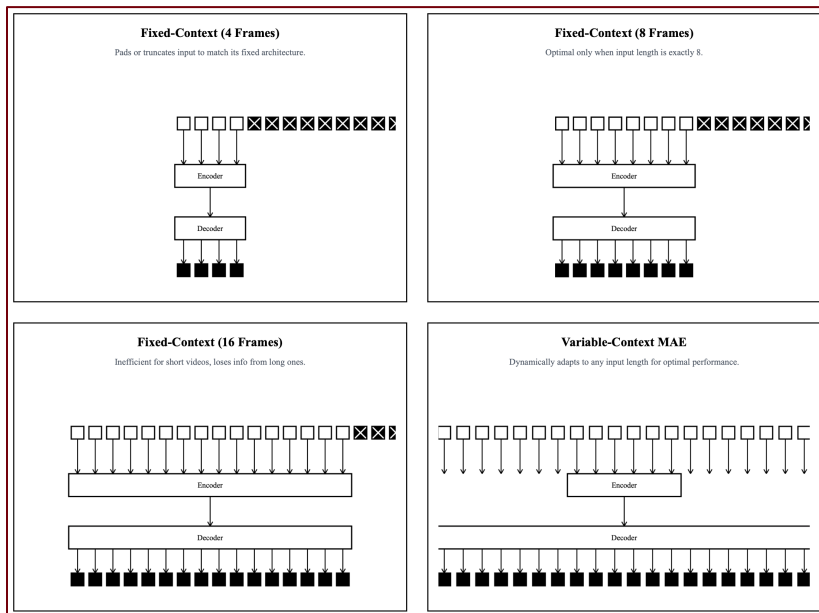
Project Overview – Three Goals

- Temporal masked auto-encoding for radar prediction
 - Variable context
 - Pulse length and Gain
- Distributed training infrastructure for multi-billion parameter models
- Classical signal processing fork
 - DBSCAN/ST-DBSCAN
 - CFAR(not really)



Masked Auto-Encoding – Attempts

- Variable context T-MAE model
- Single Frame MAE
 - Pulse Triplet model
 - Gain Triplet model



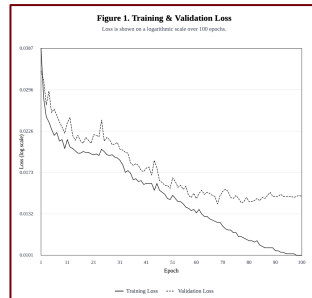
Triplet Masked Auto-Encoding – Results

- Unable to render small objects.

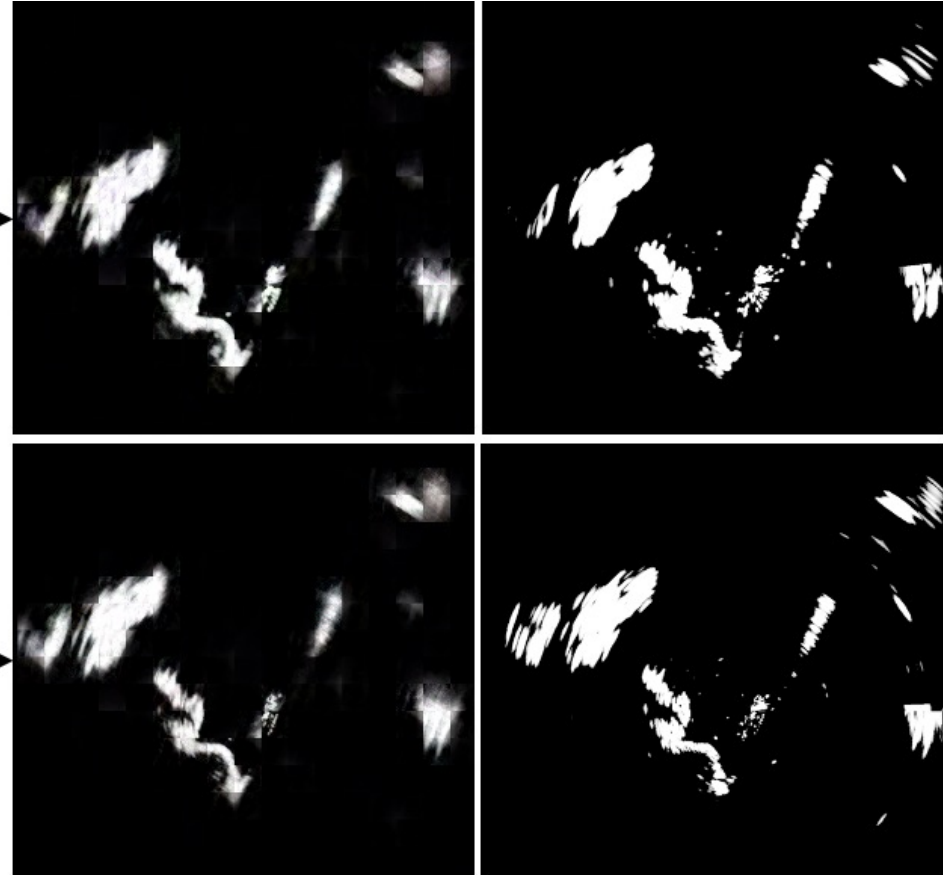
Input



Model



Non-augmented pulse



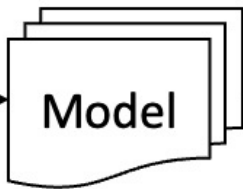
Triplet Masked Auto-Encoding – Results

- Added augmentations... made results worse??

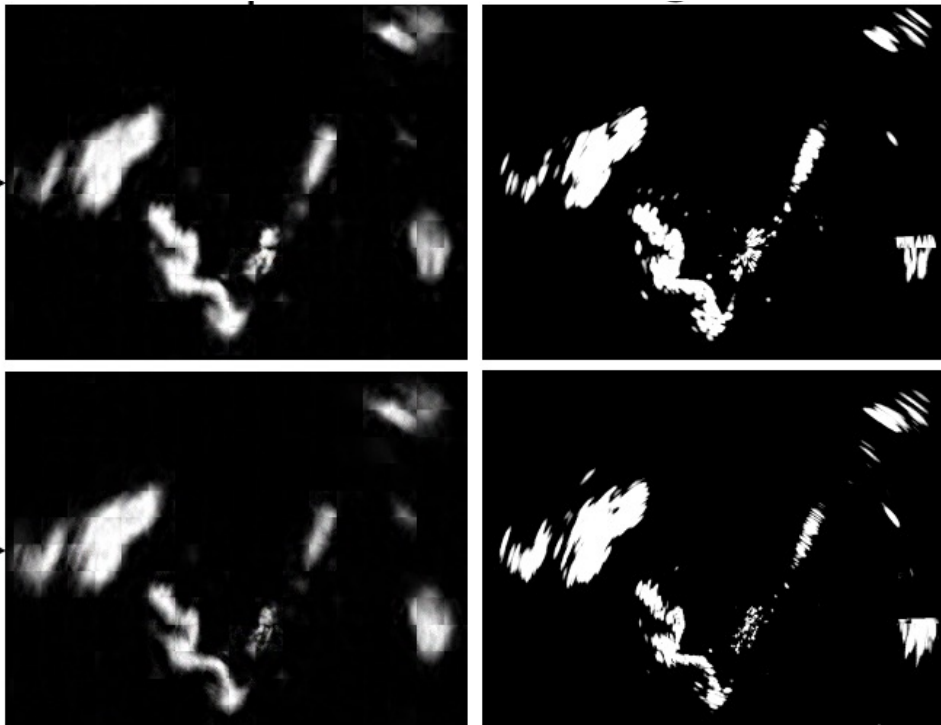
Input



Model



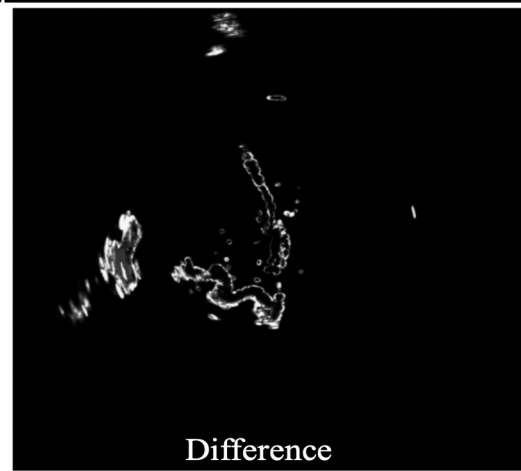
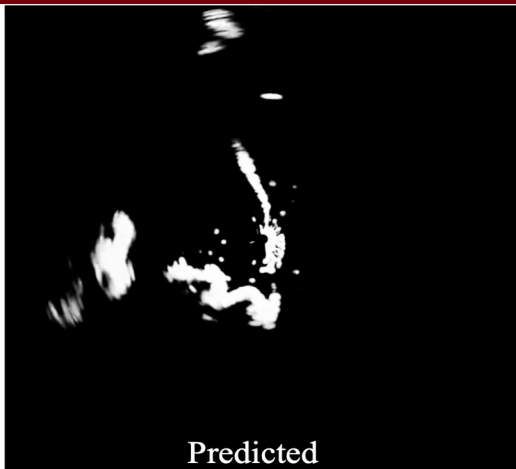
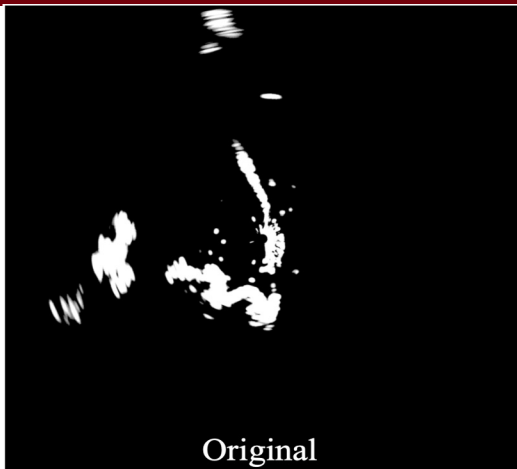
Augmented pulse



Triplet Masked Auto-Encoding – Results

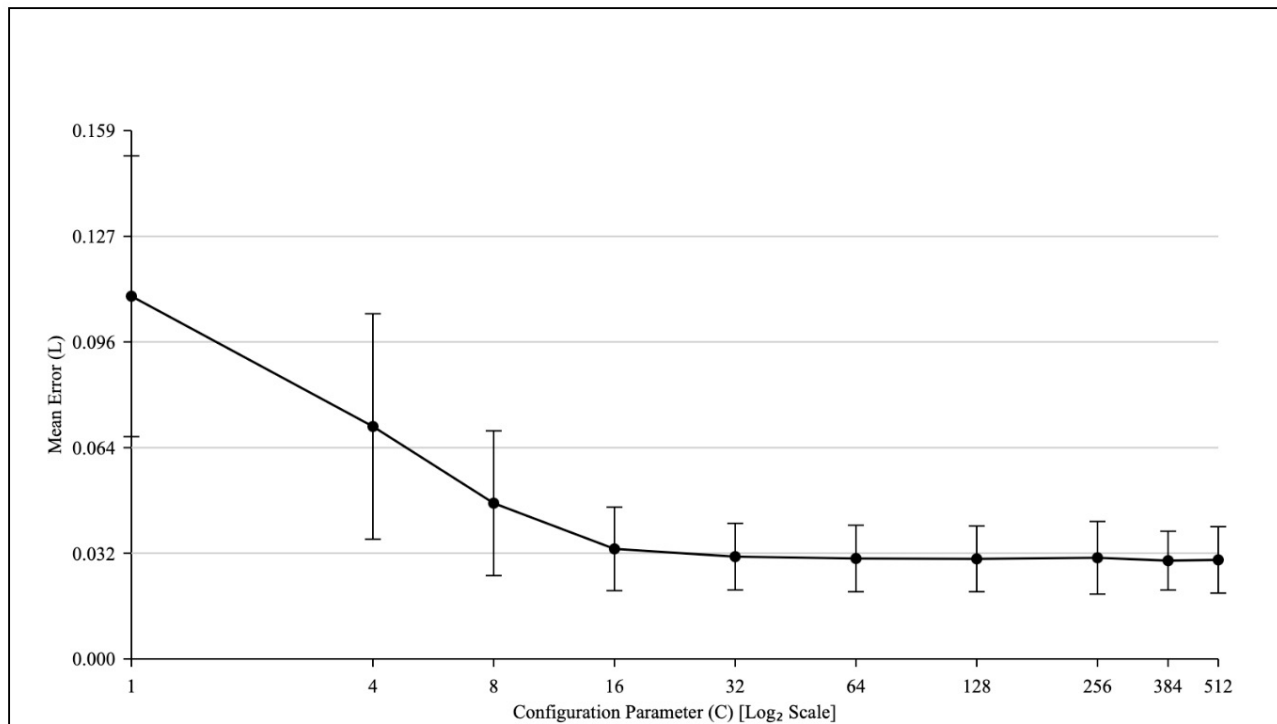
Gain
prediction
works very
well.
But is it good
enough? I
don't think so
yet

(t-MAE, not
regular MAE)



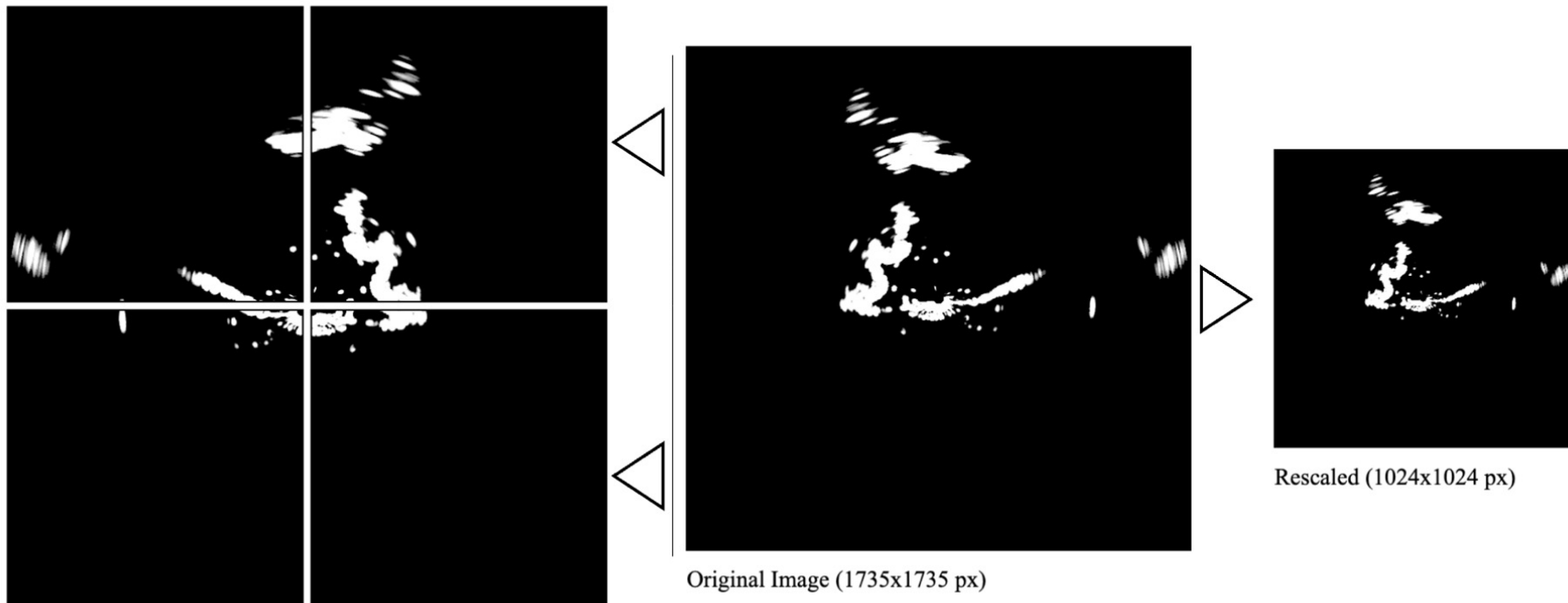
Variable Context Masked Auto-Encoding – Results

- The Variable Context model required a LOT more GPU power, thus was not trained as long.
- Performance increases dropped off with more context though
- Trained for few epochs, performed worse than MAE. Might need more testing



Data Augmentation – summary

Data upscaling/downscaling testing

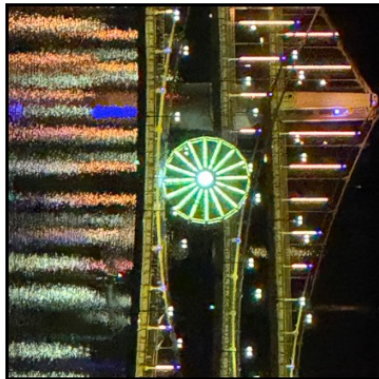


Data Augmentation – summary

Horizontal Flip



Rotation



Gaussian Noise



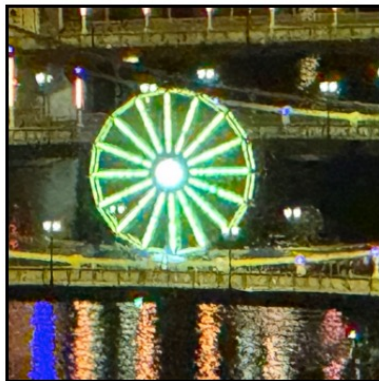
Brightness



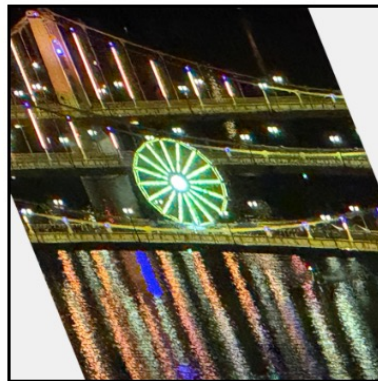
Grayscale



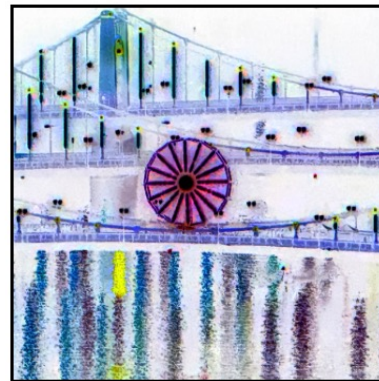
Random Crop



Shear



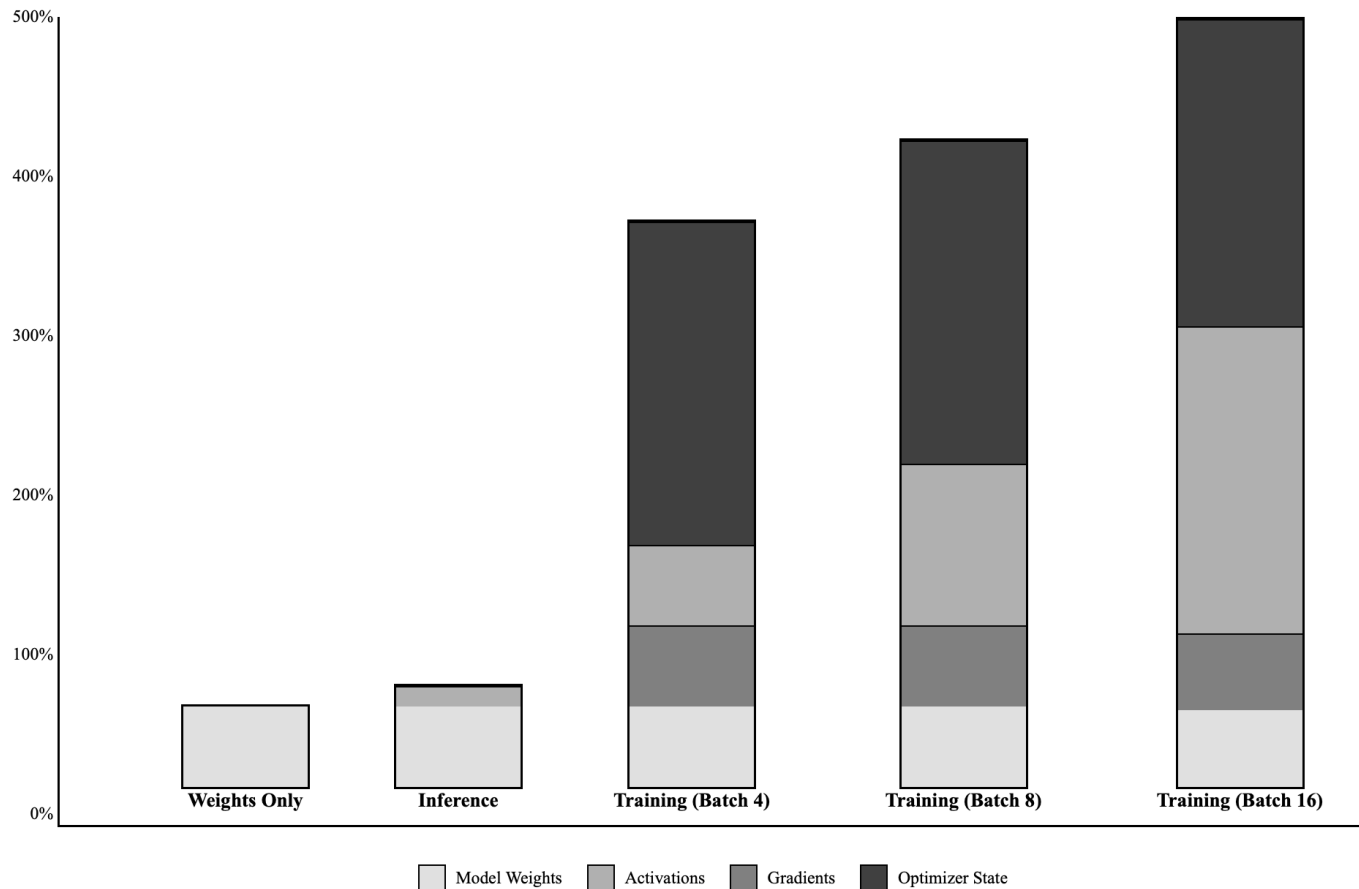
Contrast



Distributed Training Infrastructure – summary

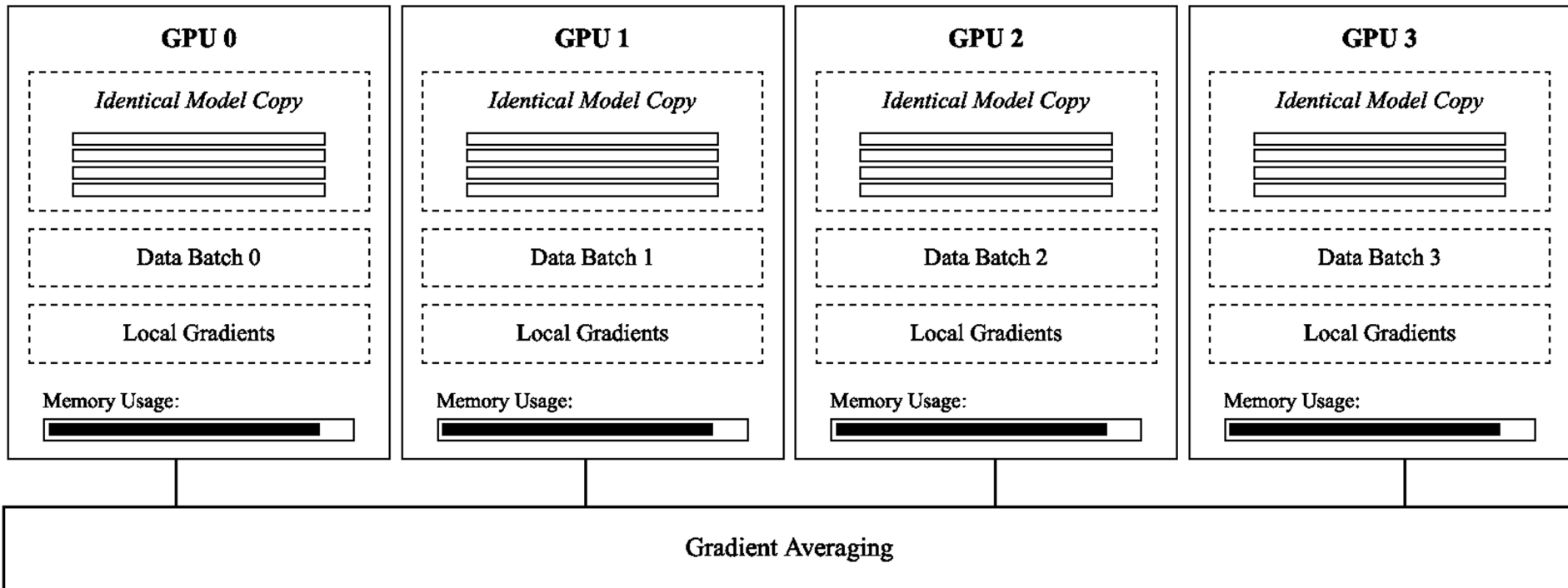
Training size is a lot bigger than the model weights. Obvious in hindsight.

Therefore, methods are developed to split models across multiple GPUs (or sharding)



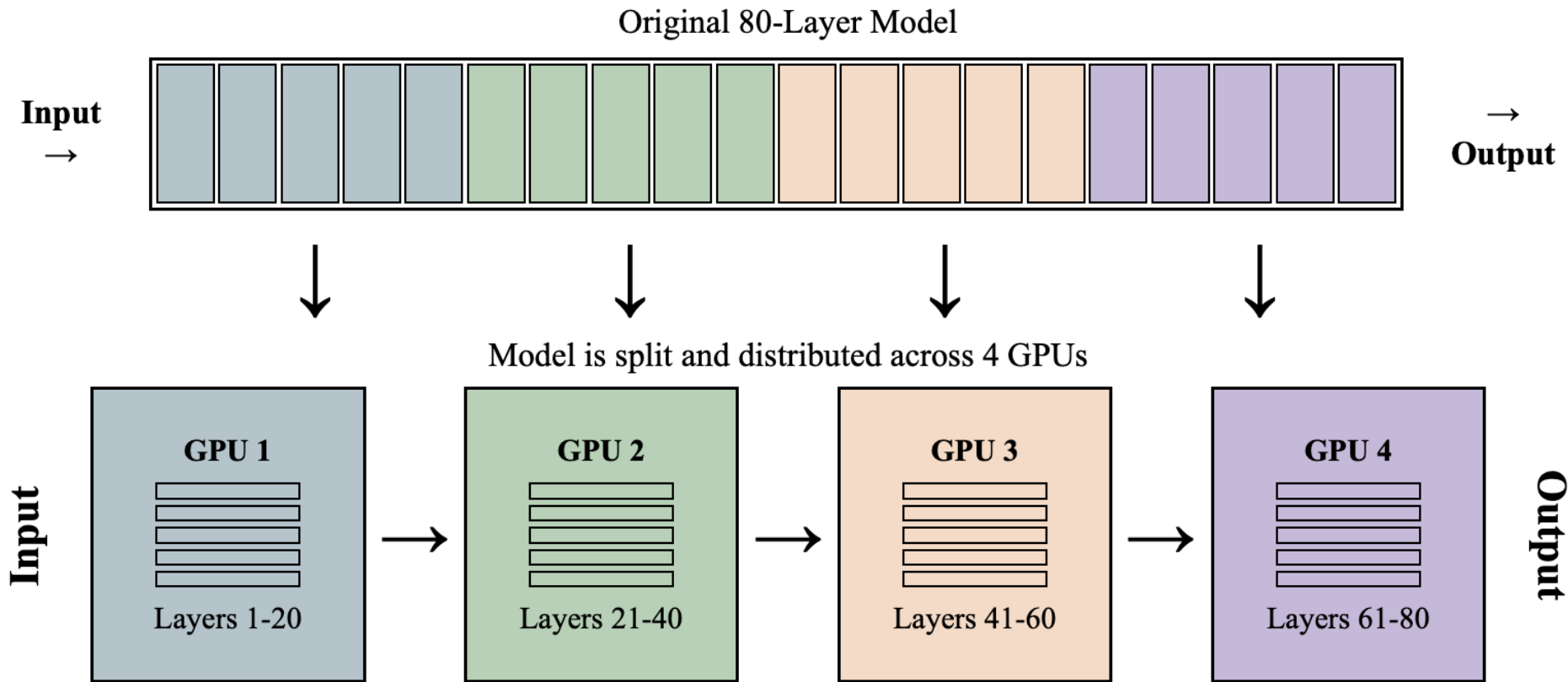
Distributed Training Infrastructure – Data parallel

Data parallelism just means we can process more batches or larger batches faster. Does not reduce model size on GPU.



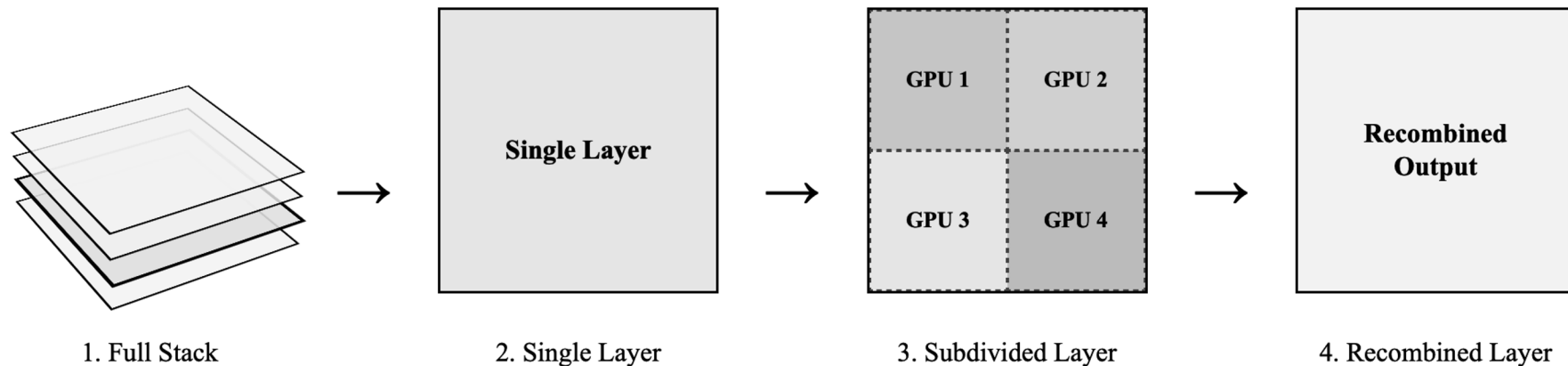
Distributed Training Infrastructure – Model parallel

Model parallelism allows for models to be split layer by layer onto GPUs.



Distributed Training Infrastructure – Tensor parallel

Tensor parallelism just split models on a per-layer basis. Best for wide models.



Distributed Training Infrastructure – Model parallel

Problem with Model parallelism is the GPU utilization being $1/N$ where N is the number of GPUs.

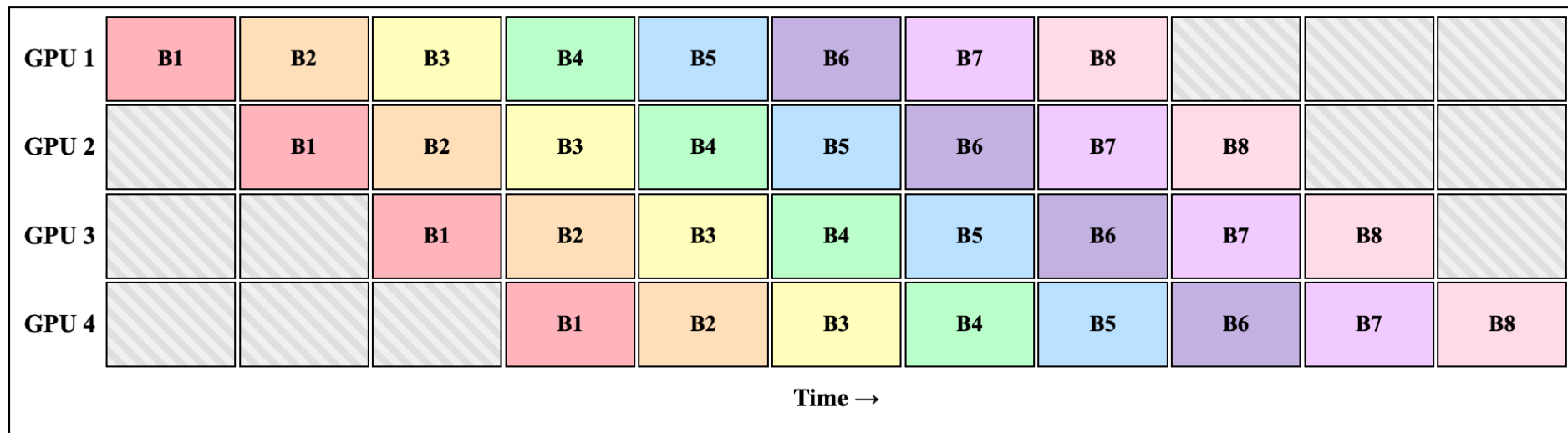
This means for a 4 GPU setup, the GPUs are only used 25% of the time

Processing Timeline

	Timestep 1	Timestep 2	Timestep 3	Timestep 4
GPU 1	Active	Idle	Idle	Idle
GPU 2	Idle	Active	Idle	Idle
GPU 3	Idle	Idle	Active	Idle
GPU 4	Idle	Idle	Idle	Active

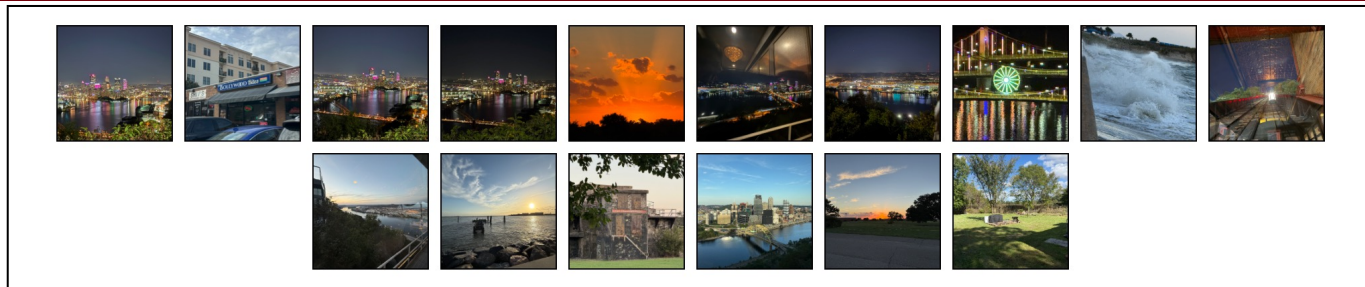
Distributed Training Infrastructure – Model parallel

Pipeline parallelism fixes this by splitting batches into micro batches so that each Gpu can begin processing the second micro batch when the second starts the next set of layers.



Distributed Training Infrastructure – Micro-Batch

The Micro batch
Process

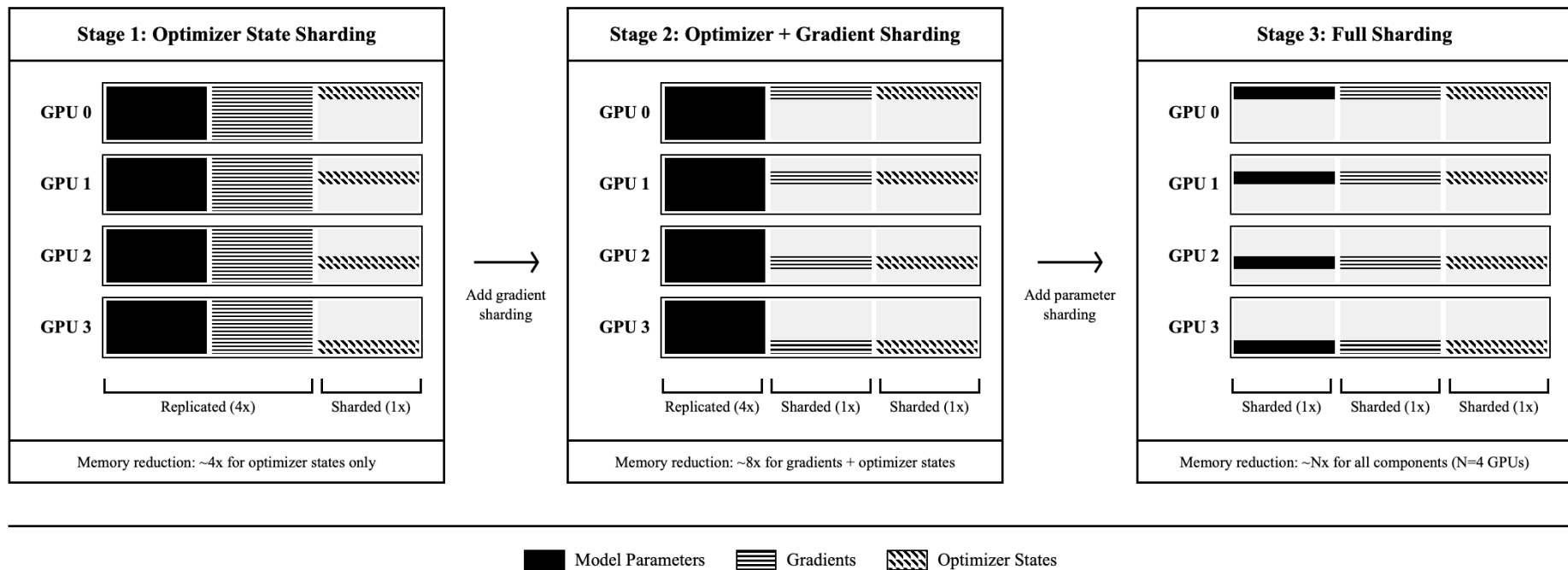


Split Micro-Batches



Distributed Training Infrastructure – DeepSpeed

I ended up deciding to be smart and use a pre-built library to do sharding for me; combined with HF accelerate to allow for quick modifications in the future.

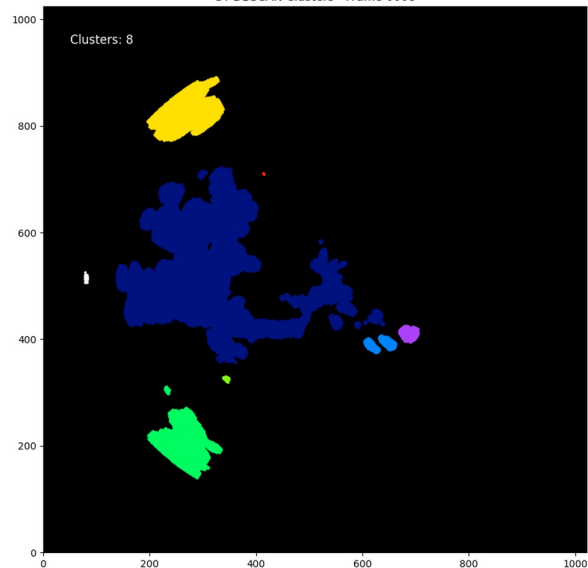


Classical Processing – ST DBSCAN and DBSCAN

- Sam developed this.
- Tracking IDs persist across frames
- Validation confirms accuracy
- **Processing Time:** ~1900 sec for 20 frames -> 500 sec for 20 frames(GPU acceleration)

DBSCAN Results

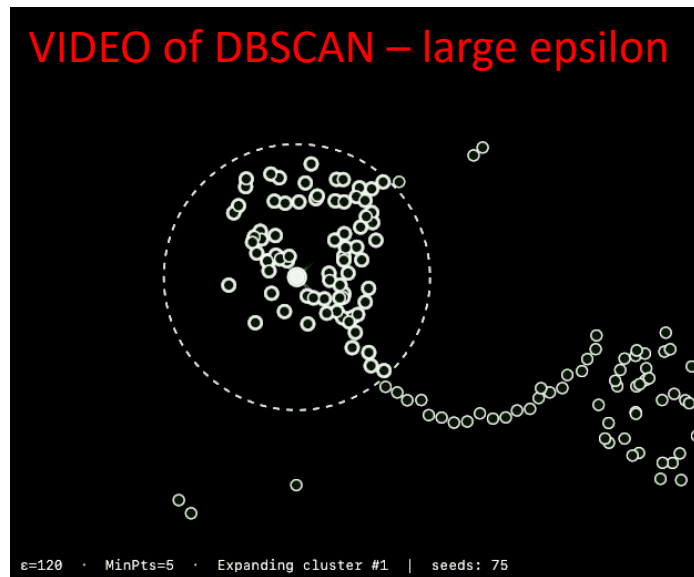
ST-DBSCAN Clusters - Frame 0008



Raw Radar Data

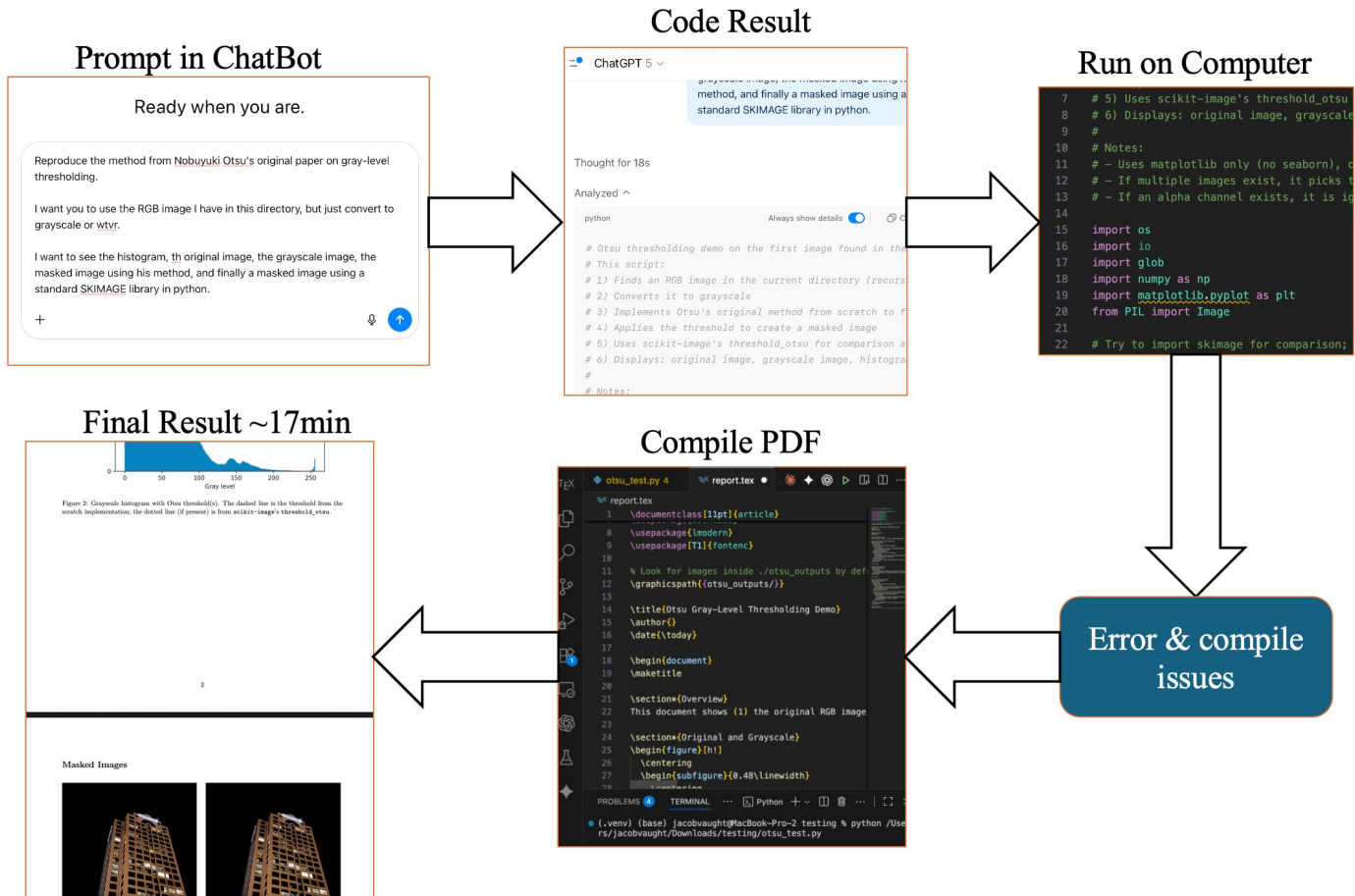


VIDEO of DBSCAN – large epsilon



AI agentic coding for undergrads

- Using AI agent tools to speed up coding and onboarding
- Making them use Google Gemini and Alibaba Qwen Code
- Great progress so far; ST-DBSCAN, CFAR implementation, etc, etc.



AI agentic coding for undergrads

- Recently purchased a team account for the undergrads to use to help speed up the work (and increase quality)

Single Prompt

```
((base) jacobvaught@MacBook-Pro-2 testing % codex --dangerously-bypass-approvals-and-sandbox --search

>_ OpenAI Codex (v0.46.0)

model: gpt-5-codex high /model to change
directory: ~/Downloads/testing

To get started, describe a task or try one of these commands:

/init - create an AGENTS.md file with instructions for Codex
/status - show current session configuration
/approvals - choose what Codex can do without approval
/model - choose what model and reasoning effort to use
/review - review any changes and find issues

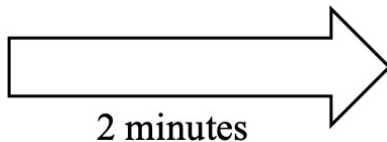
> Reproduce the method from Nobuyuki Otsu's original paper on gray-level thresholding.

use the RGB image I have in this directory, but just convert to grayscale or wtvr.

I want to see the histogram, th original image, the grayscale image, the masked image using his method, and finally a masked image using a standard SKIMAGE library in python

final deliverable needs to be a latex report that is detailed enough to explain the process using the paper, and should include a diagram made in tiki and should show the png outputs form the code in the report. when compiling, delete all intermediate files; should only have a pdf and tex file.

100% context left
```



Final Result

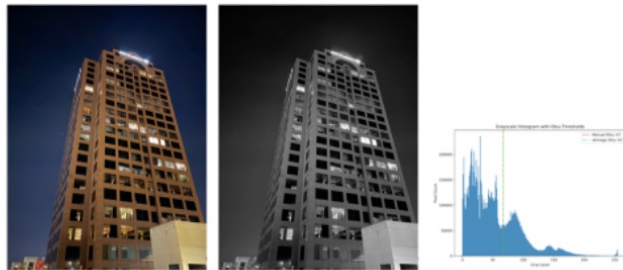


Figure 2: Left: original RGB image. Middle: BT.601 grayscale conversion. Right: grayscale histogram with manual (dashed) and library (dash-dotted) thresholds marked.



Figure 3: Left: binary mask from manual Otsu implementation. Middle: manual mask applied to RGB image. Right: mask applied using scikit-image's Otsu threshold; results coincide with the manual reproduction.

Questions?

J.C. Vaught

Department of Mechanical Engineering
University of South Carolina